

SPORTS STATS PROJECT (REPORT)

PHASE 1: SETTING UP

STEP 1: Database Creation

We first create a database named Sports_stats, create the member tables (players, matches, player_stats) and implement the relationships as follows.

```
| players | CREATE TABLE `players` (  
  `player_id` varchar(100) NOT NULL,  
  `first_name` text DEFAULT NULL,  
  `last_name` text DEFAULT NULL,  
  `team` text DEFAULT NULL,  
  `position` text DEFAULT NULL,  
  PRIMARY KEY (`player_id`)
```

```
| matches | CREATE TABLE `matches` (  
  `match_id` varchar(100) NOT NULL,  
  `date` date DEFAULT NULL,  
  `home_team` text DEFAULT NULL,  
  `away_team` text DEFAULT NULL,  
  PRIMARY KEY (`match_id`)
```

```
| player_stats | CREATE TABLE `player_stats` (  
  `stat_id` varchar(100) NOT NULL,  
  `match_id` varchar(100) NOT NULL,  
  `player_id` varchar(100) NOT NULL,  
  `goals` int(11) DEFAULT NULL,  
  `assists` int(11) DEFAULT NULL,  
  `fouls` int(11) DEFAULT NULL,  
  PRIMARY KEY (`stat_id`, `match_id`, `player_id`),  
  KEY `match_id` (`match_id`),  
  KEY `player_id` (`player_id`),  
  CONSTRAINT `player_stats_ibfk_1` FOREIGN KEY (`match_id`) REFERENCES `matches` (`match_id`),  
  CONSTRAINT `player_stats_ibfk_2` FOREIGN KEY (`player_id`) REFERENCES `players` (`player_id`)
```

STEP 2: Inserting data in our tables

Since our tables have hundreds of records manual insertion would take forever so instead you can use a python code that reads from the csv files and writes in our tables.

To create a python database connection:

```
write.py x
1 import pandas as pd
2 import mysql.connector
3
4 #creating the connection
5 connection = mysql.connector.connect(host="localhost",user="root",password="",database="sports_stats")
```

Then we read from the csv files and write in our tables:

```
#reading the data from the csv files
matches=pd.read_csv("matches.csv")
players=pd.read_csv("players.csv")
player_stats=pd.read_csv("player_stats.csv")

#writing into the matches table
for _, row in matches.iterrows():
    cursor.execute("""insert into matches values(%s,%s,%s,%s)""",(row['match_id'],row['date'],row['home_team'],row['a

#writing into the matches players
for _, row in players.iterrows():
    cursor.execute("""insert into players values(%s,%s,%s,%s,%s)""",(row['player_id'],row['first_name'],row['last_name'],row['age'],row['position'])

#writing into the matches player_stats
for _, row in player_stats.iterrows():
    cursor.execute("""insert into player_stats values(%s,%s,%s,%s,%s,%s,%s)""",(row['stat_id'],row['match_id'],row['player_id'],row['goals'],row['assists'],row['yellow_cards'],row['red_cards'])
```

Then finally we save changes and terminate the connection:

```
#saving and closing the connection
connection.commit()
cursor.close()
connection.close()

print("Data successfully entered into all tables")
```

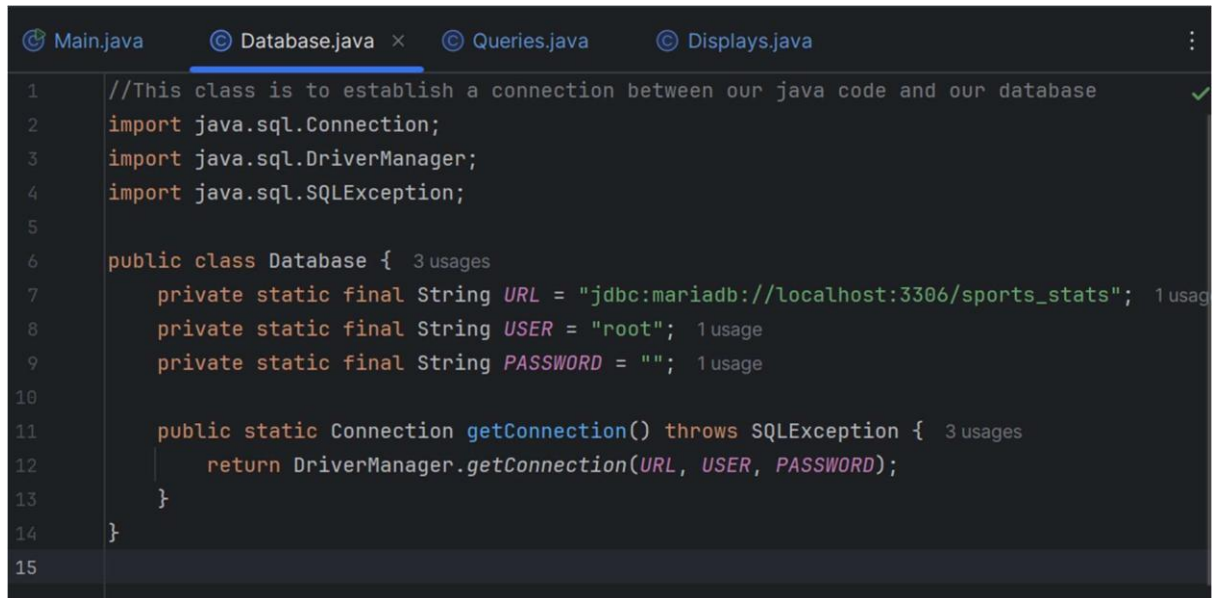
PHASE 2: JAVA CODES

For the structure of our project, we subdivided the tasks into four main classes which are:

- Database.java (To create a java-database connection)
- Queries.java (To run the queries that give us the desired results and store them in lists)
- Displays.java (Use the lists with desired records to draw charts, graphs and heatmaps)
- Analytics.java (To create a GUI and display all the analytical graphs and charts)
- Tables.java (To read all records of the database and display them in tables)
- Insertion.java (To write new data to the tables)
- Main.java (To create a frame with three options. Updating the tables, viewing the tables or graphing results)

DATABASE.JAVA

We import all the libraries needed to create the connection then we create a function getConnection() that will be called every time we need to execute and run queries



```
1 //This class is to establish a connection between our java code and our database ✓
2 import java.sql.Connection;
3 import java.sql.DriverManager;
4 import java.sql.SQLException;
5
6 public class Database { 3 usages
7     private static final String URL = "jdbc:mariadb://localhost:3306/sports_stats"; 1 usag
8     private static final String USER = "root"; 1 usage
9     private static final String PASSWORD = ""; 1 usage
10
11     public static Connection getConnection() throws SQLException { 3 usages
12         return DriverManager.getConnection(URL, USER, PASSWORD);
13     }
14 }
15
```

QUERIES.JAVA

In this class the main task is to create functions that do the following:

- Call the connection from database class to start a connection
- Creating and execute the queries for: table insertion, reading from the table, data analysis
- Store the query results into an array list
- Return that array list

We will make functions for the queries we need to execute(getTopScorers(), getTeamPerformance(), getPlayerAverages() , showPlayers(), showMatches(), showPlayerStats(), insertPlayer(), insertMatch(), insertStat()).

ANALYTICS FUNCTIONS

```
© Main.java    © Database.java    © Queries.java ×    © Displays.java

1 //We create functions to return a list of the desired query results
2 import java.sql.*;
3 import java.util.*;
4
5 public class Queries { 3 usages
6
7 @    public static List<Object[]> getTopScorers() throws SQLException { 1 usage
8     //we store the query in a string variable
9     String query = ""
10         SELECT s.player_id, p.first_name, p.last_name, SUM(s.goals) AS total_goals
11         FROM player_stats AS s
12         JOIN players AS p ON s.player_id = p.player_id
13         GROUP BY p.player_id
14         ORDER BY total_goals DESC;
15     """;
16
17     //we create a list that will hold every row as an object
18     List<Object[]> data = new ArrayList<>();
19
20     //We establish a connection from the database class and execute the query
21     try (Connection con = Database.getConnection();
22         Statement stmt = con.createStatement();
23         ResultSet rs = stmt.executeQuery(query)) {
24         //Use of a while loop to record each row of the resultset into our list
25         while (rs.next()) {
26             data.add(new Object[]{
27                 rs.getString( columnLabel: "player_id"),
28                 rs.getString( columnLabel: "first_name")+" "+rs.getString( columnLabel: "last_name"),
29                 rs.getInt( columnLabel: "total_goals")
30             });
31         }
32     }
33     return data;
34 }
```

VIEWING FUNCTIONS

```
//Functions that will be used for table displays
public static void showPlayers(DefaultTableModel model) throws SQLException { 1 usage
    String sql = "Select * from players";
    try (Connection con = Database.getConnection();
        Statement stmt = con.createStatement();
        ResultSet rs = stmt.executeQuery(sql);
    ) {
        while (rs.next()) {
            model.addRow(new Object[]{
                rs.getString( columnLabel: "player_id"),
                rs.getString( columnLabel: "first_name"),
                rs.getString( columnLabel: "last_name"),
                rs.getString( columnLabel: "team"),
                rs.getString( columnLabel: "position")
            });
        }
    }
}
```

INSERTING FUNCTIONS

```
//Functions used for data insertion
public static void insertPlayer(String id,String fname,String lname,String team,String position) throws SQLException {
    String sql="Insert into players values(?,?, ?,?,?)";

    try {Connection con = Database.getConnection();
    PreparedStatement stmt = con.prepareStatement(sql);
    stmt.setString( parameterIndex: 1,id);
    stmt.setString( parameterIndex: 2,fname);
    stmt.setString( parameterIndex: 3,lname);
    stmt.setString( parameterIndex: 4,team);
    stmt.setString( parameterIndex: 5,position);
    stmt.executeUpdate();
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
    JOptionPane.showMessageDialog( parentComponent: null, message: "Player has been inserted successfully!");
}
```

TABLES.JAVA

In this class We create a function called show tables that will create a frame and create three tabs one for each table in our database. Then we create a table model and of the same format as the table we want to create. We create a table and pass the model to it. Afterwards, we create a scrolling pane to enable us to scroll when we have large volumes of data. Finally, we add the scroll pane to the tabs and add the tabs to the frame

```
© Tables.java × © Analytics.java © Database.java © Insertion.java © Queries.java
1 import javax.swing.*;
2 import javax.swing.table.DefaultTableModel;
3 import java.awt.*;
4 import java.sql.SQLException;
5
6 public class Tables { 1 usage
7     @ public static JFrame showTables() throws SQLException { 1 usage
8         JFrame frame=new JFrame( title: "Tables");
9         frame.setSize( width: 1000, height: 700);
10
11         DefaultTableModel model1=new DefaultTableModel();
12
13         model1.addColumn( columnName: "player_id");
14         model1.addColumn( columnName: "first_name");
15         model1.addColumn( columnName: "last_name");
16         model1.addColumn( columnName: "team");
17         model1.addColumn( columnName: "position");
18
19         JTable players=new JTable(model1);
20
21         Queries.showPlayers(model1);
22
23         JScrollPane scroll=new JScrollPane(players);
24 }
```

```

        JTabbedPane tabs=new JTabbedPane();

        tabs.addTab( title: "Players",scroll);
        tabs.addTab( title: "Matches",scroll2);
        tabs.addTab( title: "Player Stats",scroll3);

        frame.add(tabs, BorderLayout.CENTER);

        frame.setVisible(true);
        return frame;
    }
}

```

INSERTION.JAVA

In this class we create a function that returns a frame with three tabs. Each tab represents one of our tables in the data base and has text fields for the values. After receiving the values pressing the insert button calls functions from queries.java that allows us to insert in a given table in the database.

```

import javax.swing.*;
import java.awt.*;
import java.sql.SQLException;

public class Insertion { 1 usage
    public static JFrame insertData(){ 1 usage
        JFrame frame = new JFrame( title: "Insertion Window");
        frame.setSize( width: 1000, height: 700);

        JPanel playersPanel = new JPanel();
        playersPanel.setLayout(new GridLayout( rows: 6, cols: 2, hgap: 10, vgap: 60));

        playersPanel.add(new JLabel( text: "Player ID:"));
        JTextField playerID = new JTextField();
        playersPanel.add(playerID);

        playersPanel.add(new JLabel( text: "First Name:"));
        JTextField firstName = new JTextField();
        playersPanel.add(firstName);

        playersPanel.add(new JLabel( text: "Last Name:"));
        JTextField lastName = new JTextField();
        playersPanel.add(lastName);

        playersPanel.add(new JLabel( text: "Team:"));
        JTextField team = new JTextField();
        playersPanel.add(team);

        playersPanel.add(new JLabel( text: "Position:"));
        JTextField position = new JTextField();
        playersPanel.add(position);
    }
}

```

```

playersPanel.add(new JLabel( text: "Position:"));
JTextField position = new JTextField();
playersPanel.add(position);

JButton p_insert = new JButton( text: "Insert");
p_insert.addActionListener( ActionEvent e -> {
    try {
        Queries.insertPlayer(playerID.getText(), firstName.getText(), lastName.getText(), team.getText(), position.getText());
    } catch (SQLException ex) {
        throw new RuntimeException(ex);
    }
});
playersPanel.add(p_insert);

```

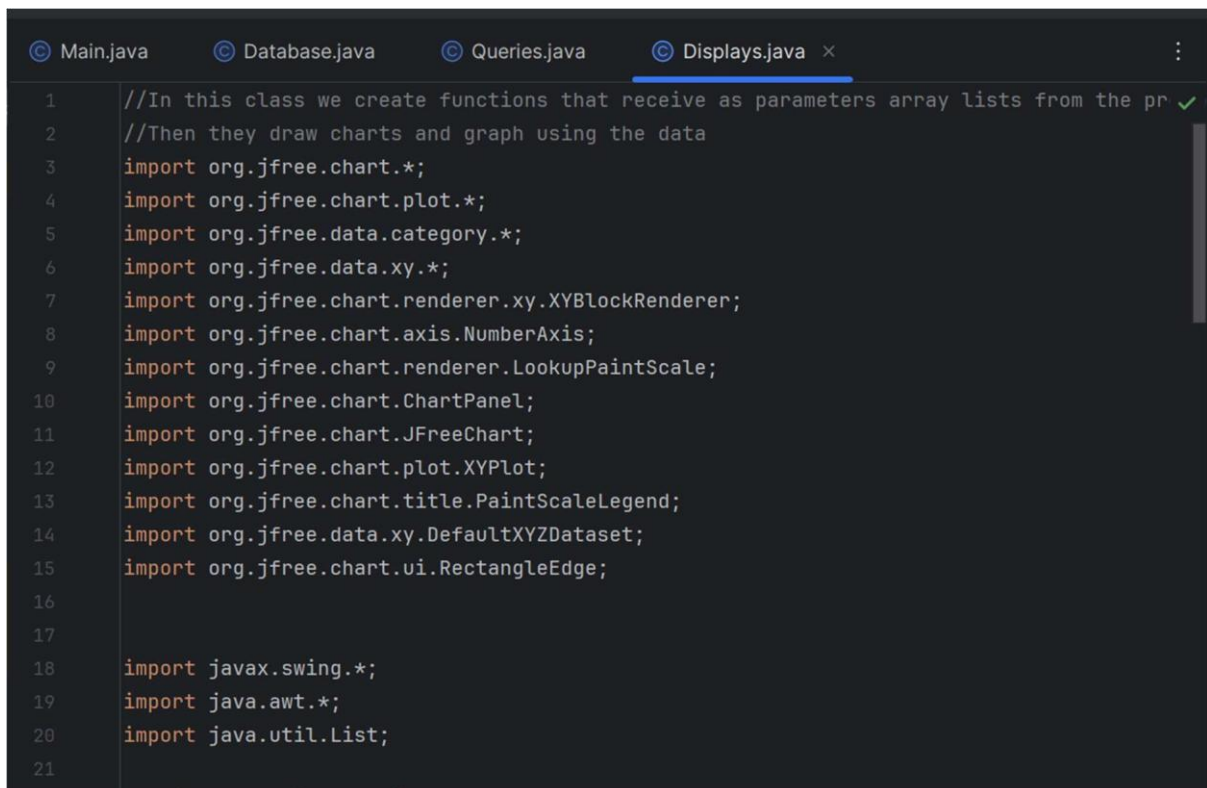
DISPLAYS.JAVA

This class is made up of three functions to draw each of the displays we need make.

- A bar chart for top scorers (createBarChart())
- A line chart for team performances (createLineChart())
- A heatmap for player contributions (createHeatMap())

These functions will receive the array lists containing results from the queries as a parameter. And they will return the graphs.

NOTE! : Make sure to import all the graphing libraries needed.



```

Main.java Database.java Queries.java Displays.java ×
1 //In this class we create functions that receive as parameters array lists from the pr ✓
2 //Then they draw charts and graph using the data
3 import org.jfree.chart.*;
4 import org.jfree.chart.plot.*;
5 import org.jfree.data.category.*;
6 import org.jfree.data.xy.*;
7 import org.jfree.chart.renderer.xy.XYBlockRenderer;
8 import org.jfree.chart.axis.NumberAxis;
9 import org.jfree.chart.renderer.LookupPaintScale;
10 import org.jfree.chart.ChartPanel;
11 import org.jfree.chart.JFreeChart;
12 import org.jfree.chart.plot.XYPlot;
13 import org.jfree.chart.title.PaintScaleLegend;
14 import org.jfree.data.xy.DefaultXYZDataset;
15 import org.jfree.chart.ui.RectangleEdge;
16
17
18 import javax.swing.*;
19 import java.awt.*;
20 import java.util.List;
21

```

createBarChart()

Since charts allow entry through datasets. We create a dataset, and, on the x-axis, we will be adding the IDs of the players and comparing them against the total goals scored on the y-axis. Then we create a barchart and label it and finally pass the dataset to it. The final step is returning the chart.

```
public class Displays { 3 usages

    public static JPanel createBarChart(List<Object[]> data) { 1 usage
        //We create a dataset that will hold the data for both axes of the graph
        //we read the values we need from the data arraylist
        DefaultCategoryDataset dataset = new DefaultCategoryDataset();
        for (Object[] row : data) {
            dataset.addValue((Number) row[2], rowKey: "Goals", (String) row[0]);
        }

        //We create the barchart and label the title and axes
        JFreeChart chart = ChartFactory.createBarChart(
            title: "Top Scorers", categoryAxisLabel: "Player", valueAxisLabel: "Total Goals", dataset,
            PlotOrientation.VERTICAL, legend: false, tooltips: true, urls: false);

        return new ChartPanel(chart);
    }
}
```

createLineChart()

Similarly to the barchart we will need to create a dataset. We will be displaying every team (row [0]) against their wins (row [1]), draws (row [2]), losses (row [3]) and goal difference (row [4]). Then again we create the line chart, label it and pass the dataset to it then finally return it.

```
39 //Same logic as in the bar graph above
40 @ public static JPanel createLineChart(List<Object[]> data) { 1 usage
41     DefaultCategoryDataset dataset = new DefaultCategoryDataset();
42     for (Object[] row : data) {
43         dataset.addValue((Number) row[1], rowKey: "Wins", (String) row[0]);
44         dataset.addValue((Number) row[2], rowKey: "Draws", (String) row[0]);
45         dataset.addValue((Number) row[3], rowKey: "Losses", (String) row[0]);
46         dataset.addValue((Number) row[4], rowKey: "Goal_Difference", (String) row[0]);
47     }
48
49     JFreeChart chart = ChartFactory.createLineChart(
50         title: "Team Performance", categoryAxisLabel: "Team", valueAxisLabel: "Results", dataset,
51         PlotOrientation.VERTICAL, legend: true, tooltips: true, urls: false);
52
53     return new ChartPanel(chart);
54 }
```

createHeatMap()

This one is a bit more complex, but the initial steps are similar. Here our heatmap will be determined by three variables (hence we will use an XYZ Dataset):

- X-axis with the player indices/IDs to identify them
- Y-axis with the ratings of each player (formula = (goals + assists – fouls)/3)
- Z-axis this the metric used for the colouring (the renderer)

And since I want the heatmap to be according to player contributions I will make $z=y$.

Then we create a multi-dimensional array to hold each player's index, and rating and renderer.

Then we loop through the query result and add each array to our dataset

```
public static JPanel createHeatMap(List<Object[]> data) { 1 usage
    // Create XYZ dataset where z = y
    DefaultXYZDataset dataset = new DefaultXYZDataset();
    double[][] values = new double[3][data.size()]; // x, y, z

    int i = 0;
    for (Object[] row : data) {
        // Compute the metric
        double avg = (((Number) row[1]).doubleValue() +
            ((Number) row[2]).doubleValue() -
            ((Number) row[3]).doubleValue()) / 3.0;

        values[0][i] = i + 1; // X: player index
        values[1][i] = avg;    // Y: metric value
        values[2][i] = avg;    // Z: same value for color
        i++;
    }
    dataset.addSeries( seriesKey: "Player Metrics", values);
}
```

Then we label the axes and define our rendering/colouring scheme.

```

// Axes
NumberAxis xAxis = new NumberAxis( label: "Player Index");
NumberAxis yAxis = new NumberAxis( label: "Average Metric");

// Renderer setup
XYBlockRenderer renderer = new XYBlockRenderer();
renderer.setBlockWidth(1.0);
renderer.setBlockHeight(0.05);

// Paint scale (color mapping)
LookupPaintScale paintScale = new LookupPaintScale( lowerBound: -0.5, upperBound: 2.0, Color.GRAY);
paintScale.add( value: -0.5, Color.MAGENTA); //Worst
paintScale.add( value: 0.15, Color.RED); //Bad
paintScale.add( value: 0.45, Color.ORANGE); //Average
paintScale.add( value: 0.75, Color.YELLOW); //Good
paintScale.add( value: 1.05, Color.GREEN); //Elite

renderer.setPaintScale(paintScale);

```

The final step is to plot our heatmap and put it on a free chart and return it.

```

// Plot
XYPlot plot = new XYPlot(dataset, xAxis, yAxis, renderer);
plot.setBackgroundPaint(Color.WHITE);

// Add legend to explain color scale
PaintScaleLegend legend = new PaintScaleLegend(paintScale, new NumberAxis( label: "Metric Value"));
legend.setPosition(RectangleEdge.RIGHT);

// Chart
JFreeChart chart = new JFreeChart(
    title: "Player Contributions Heatmap",
    JFreeChart.DEFAULT_TITLE_FONT,
    plot,
    createLegend: false
);
chart.addSubtitle(legend);

return new ChartPanel(chart);
}

```

ANALYTICS.JAVA

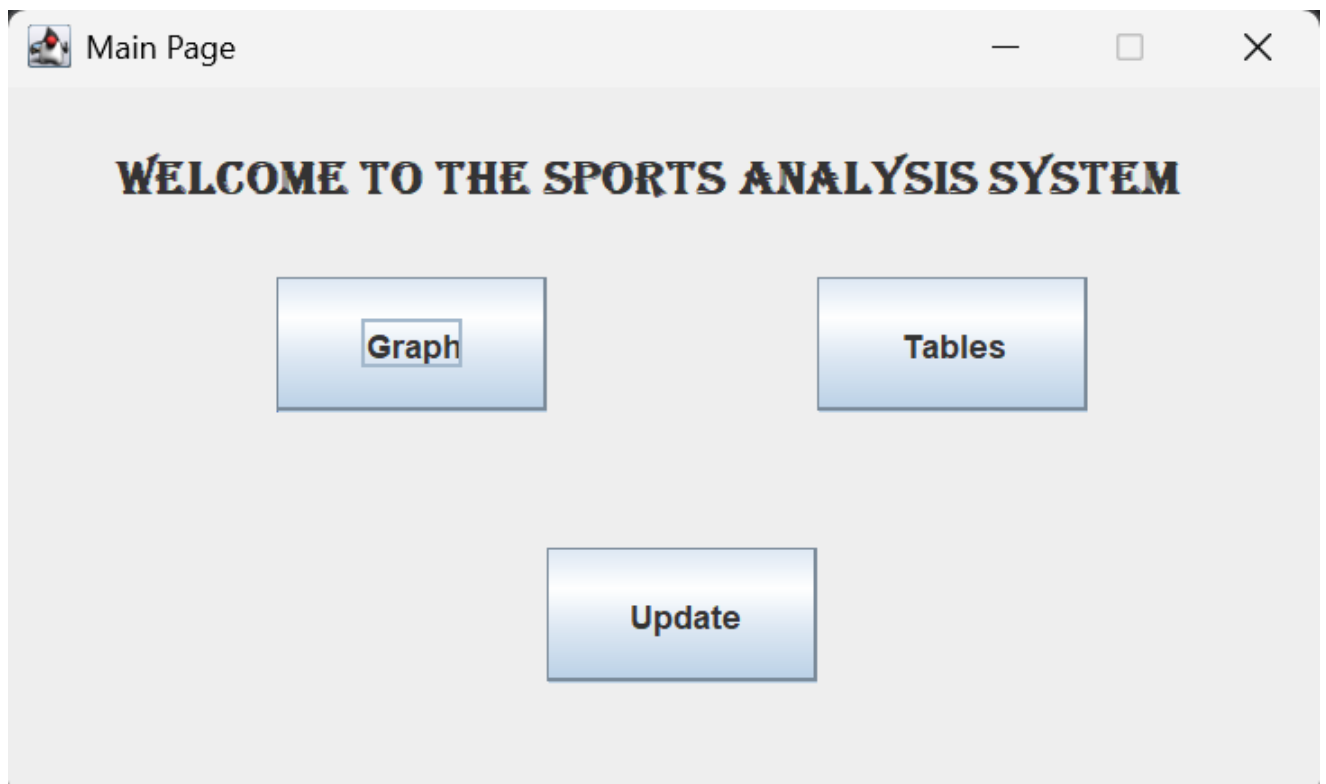
In the main we create the frame for our Graphical User Interface (GUI) label it, define its size, ...

Then we create a tabbed pane because we want to display each analysis on a different tab.

And finally, we label each tab and call the graphing functions from displays.java class

```
© Main.java × © Database.java © Queries.java © Displays.java
1 import javax.swing.*;
2 import java.awt.*;
3 import java.sql.SQLException;
4
5 public class Main {
6     public static void main(String[] args) throws SQLException {
7         //We create a frame for our GUI and initialize it's title
8         JFrame frame = new JFrame( title: "Sports Stats Dashboard");
9         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
10        frame.setSize( width: 1000, height: 700);
11
12        //Since we want multiple tabs for each our data analytics graph we create a tabbed pane
13        JTabbedPane tabs = new JTabbedPane();
14
15        //We add tabs and in each tab we add a title and call functions to display charts
16        tabs.addTab( title: "Top Scorers", Displays.createBarChart(Queries.getTopScorers()));
17        tabs.addTab( title: "Team Performance", Displays.createLineChart(Queries.getTeamPerformance()));
18        tabs.addTab( title: "Player Ratings", Displays.createHeatMap(Queries.getPlayerAverages()));
19
20        frame.add(tabs, BorderLayout.CENTER);
21        frame.setVisible(true);
22    }
23
24 }
```

OUTPUT:



Tables

Players

Matches

Player Stats

match_id	date	home_team	away_team
MS001	2025-12-25	TeamA	TeamC
MS002	2025-12-18	TeamD	TeamB
MS003	2025-01-06	TeamD	TeamA
MS004	2025-04-07	TeamB	TeamA
MS005	2025-11-18	TeamC	TeamD
MS006	2025-02-09	TeamA	TeamD
MS007	2025-09-02	TeamD	TeamC
MS008	2025-11-28	TeamD	TeamB
MS009	2025-04-18	TeamA	TeamB
MS010	2025-05-17	TeamC	TeamD
MS011	2025-11-30	TeamA	TeamC
MS012	2025-03-22	TeamC	TeamA
MS013	2025-12-04	TeamB	TeamA
MS014	2025-05-18	TeamA	TeamC
MS015	2025-05-06	TeamC	TeamB
MS016	2025-05-24	TeamD	TeamC
MS017	2025-02-01	TeamA	TeamC
MS018	2025-10-26	TeamB	TeamA
MS019	2025-01-12	TeamB	TeamC
MS020	2025-09-18	TeamD	TeamC
MS021	2025-04-27	TeamB	TeamC
MS022	2025-12-12	TeamB	TeamA
MS023	2025-07-30	TeamA	TeamC
MS024	2025-07-20	TeamB	TeamA
MS025	2025-12-23	TeamD	TeamC
MS026	2025-10-23	TeamD	TeamC
MS027	2025-03-25	TeamB	TeamC
MS028	2025-04-18	TeamD	TeamC
MS029	2025-09-24	TeamC	TeamB
MS030	2025-10-04	TeamB	TeamA
MS032	2025-12-18	Lions	Cheetas

Insertion Window

Players

Matches

PlayerStats

Match ID:

Date:

Home Team:

Away Team:

Insert

